

Reimbursements Platform Blueprint

Campo	Valor
Documento	RPC-003
Nombre	Architecture Governance
Versión	1.0
Estado	Draft
Clasificación	Blueprint
Autor	Architecture Office
Responsable	Chief Software Architect
Audiencia	Arquitectos, Líder Técnico, Líder Funcional, Desarrolladores, Revisores Técnicos, IA involucradas
Última actualización	2026-07-24

1. Propósito

Este documento define el modelo oficial de **Architecture Governance** para el Reimbursements Platform Blueprint.

Su propósito es establecer cómo deberán gobernarse las decisiones que puedan afectar:

- Arquitectura.
- Diseño.
- Integración.
- Datos.
- Seguridad.
- Operación.
- Evolución.
- Estándares.
- Calidad técnica.
- Implementación futura.

RPC-003 establece:

- Autoridades.

- Responsabilidades.
- Categorías de decisión.
- Niveles de impacto.
- Gates.
- Procesos de revisión.
- Aprobaciones.
- ADR.
- Excepciones.
- Cambios.
- Cumplimiento.
- Auditoría.
- Gestión de deuda arquitectónica.

Este documento responde:

¿Cómo se toman y controlan las decisiones arquitectónicas para preservar coherencia sin impedir la evolución del sistema?

2. Alcance

RPC-003 aplica a toda decisión con impacto arquitectónico relevante dentro del proyecto.

Incluye decisiones relacionadas con:

- Límites del sistema.
- Bounded Contexts.
- Servicios.
- Comunicación.
- APIs.
- Persistencia.
- Seguridad.
- Integraciones.
- Infraestructura.
- Deployment.
- Observabilidad.
- Testing arquitectónico.
- Dependencias.
- Standards.
- Patrones transversales.
- Excepciones.
- Evolución técnica.

Las reglas aplican independientemente de quién proponga la decisión:

- Arquitecto.
- Líder Técnico.

- Desarrollador.
 - Especialista.
 - Líder Funcional.
 - Herramienta de IA.
-

3. Fuera de Alcance

RPC-003 no define:

- Los Bounded Contexts concretos.
- Los microservicios concretos.
- Las tecnologías finales.
- Las APIs finales.
- La infraestructura final.
- El modelo de datos.
- Reglas de negocio.
- Implementaciones.

Este documento gobierna **el proceso de decisión**, no el contenido de las decisiones futuras.

4. Relación con RPC-001 y RPC-002

La jerarquía será:

RPC-001
AI Project Constitution
Define autoridad y límites de IA

↓

RPC-002
AI Collaboration Charter
Define cómo colaboran humanos e IA

↓

RPC-003
Architecture Governance
Define cómo se gobiernan decisiones arquitectónicas

RPC-003 deberá respetar los principios establecidos por ambos documentos.

5. Principio Fundamental

El gobierno arquitectónico seguirá:

Toda decisión importante debe ser proporcionalmente analizada, explícitamente aprobada y suficientemente trazable.

El objetivo no será maximizar procesos.

El objetivo será maximizar:

- Claridad.
 - Coherencia.
 - Calidad.
 - Evolución.
 - Responsabilidad.
-

6. Gobierno como Habilitador

Architecture Governance no deberá convertirse en un mecanismo burocrático.

Deberá permitir:

- Tomar decisiones con claridad.
- Reducir contradicciones.
- Detectar riesgos temprano.
- Mantener velocidad sostenible.
- Evitar deuda accidental.
- Preservar autonomía dentro de límites.

El gobierno existe para facilitar buenas decisiones.

No para centralizar innecesariamente cada detalle.

7. Principios de Gobierno

Toda actividad de gobierno deberá aplicar:

7.1 Proporcionalidad

El nivel de proceso deberá corresponder al impacto de la decisión.

7.2 Explicit Decision Making

Las decisiones relevantes deberán ser explícitas.

7.3 Traceability

Deberá poder comprenderse:

- Qué se decidió.
 - Quién aprobó.
 - Por qué.
 - Qué documentos se vieron afectados.
-

7.4 Reversibility Awareness

Las decisiones difíciles de revertir requerirán mayor análisis.

7.5 Business Alignment

La arquitectura deberá servir al negocio.

7.6 Simplicity

Se evitará introducir complejidad sin necesidad.

7.7 Evolution

Las decisiones deberán poder revisarse cuando cambie el contexto.

7.8 Distributed Responsibility

No todas las decisiones requieren intervención del Chief Software Architect.

La autoridad deberá delegarse donde sea seguro hacerlo.

8. Autoridad Arquitectónica

La autoridad final sobre decisiones arquitectónicas relevantes pertenece al equipo humano responsable.

El rol principal de custodia arquitectónica será:

Chief Software Architect

Este rol será responsable de:

- Mantener coherencia.
- Facilitar decisiones.
- Revisar decisiones de alto impacto.
- Resolver conflictos arquitectónicos.
- Mantener el Blueprint.
- Determinar cuándo se requiere ADR.
- Proteger gates.

9. Architecture Office

El término:

Architecture Office

representa la función de gobierno arquitectónico del proyecto.

No requiere necesariamente una unidad organizacional independiente.

Podrá estar compuesto por:

- Chief Software Architect.
 - Líder Técnico.
 - Especialistas convocados.
 - Líder Funcional cuando exista impacto de negocio.
-

10. Líder Técnico

El Líder Técnico podrá decidir autónomamente asuntos técnicos dentro de:

- Arquitectura aprobada.
- Standards vigentes.
- ADR vigentes.
- Límites de impacto definidos.

Deberá escalar cuando una decisión:

- Cambie arquitectura.
 - Genere una excepción.
 - Introduzca dependencia transversal.
 - Contradiga un Standard.
 - Sea difícil de revertir.
-

11. Desarrolladores

Los desarrolladores tendrán autonomía para decisiones locales de implementación cuando:

- No alteren arquitectura.
- No contradigan Standards.
- No afecten contratos externos.
- No generen dependencia transversal.
- Sean razonablemente reversibles.

El gobierno no deberá requerir ADR para decisiones locales ordinarias.

12. Líder Funcional

La autoridad funcional será necesaria cuando una decisión dependa de:

- Regla de negocio.
- Prioridad.
- Actor.
- Flujo.
- Excepción.
- Política operativa.

Arquitectura no deberá inventar ni resolver unilateralmente incertidumbre funcional.

13. Especialistas

Podrán requerirse especialistas para decisiones relacionadas con:

- Seguridad.
- Datos.
- Infraestructura.
- Compliance.
- Performance.
- Operación.

Su intervención deberá ser proporcional al riesgo.

14. Inteligencia Artificial

Una IA podrá:

- Preparar opciones.
- Analizar trade-offs.
- Generar ADR Draft.
- Revisar consistencia.
- Detectar incumplimientos.
- Identificar impacto.

No podrá:

- Aprobar decisiones.
 - Autorizar excepciones.
 - Cambiar gates.
 - Declarar cumplimiento definitivo.
-

15. Categorías de Decisión

Las decisiones podrán clasificarse conceptualmente como:

Local
Tactical
Architectural
Strategic
Constitutional

La clasificación ayudará a determinar el nivel de gobierno requerido.

16. Local Decision

Una decisión Local:

- Tiene impacto limitado.
- Es fácilmente reversible.
- No cambia contratos importantes.
- No contradice arquitectura.

Ejemplos futuros:

- Organización interna de un método.
- Naming local permitido.
- Refactorización pequeña.

Normalmente:

Developer / Technical Lead

podrá decidirla sin ADR.

17. Tactical Decision

Una decisión Tactical:

- Afecta un componente o área.
- Tiene impacto moderado.
- Puede establecer una práctica reusable.
- Puede requerir revisión técnica.

Podrá requerir:

- Líder Técnico.
- Peer Review.
- Actualización de Guide o Standard.

No necesariamente requerirá ADR.

18. Architectural Decision

Una decisión Architectural:

- Afecta múltiples componentes.
- Modifica límites.
- Define interacción.
- Introduce una dependencia importante.
- Tiene impacto de largo plazo.

Normalmente requerirá:

- Architecture Review.
 - Trade-off analysis.
 - Aprobación.
 - ADR cuando sea significativa.
-

19. Strategic Decision

Una decisión Strategic:

- Afecta la dirección tecnológica.
- Tiene alto costo de reversión.
- Cambia una baseline importante.
- Puede afectar operación u organización.

Requerirá:

- Análisis formal.
 - Alternativas.
 - Architecture Review.
 - ADR.
 - Aprobación explícita.
-

20. Constitutional Decision

Una decisión Constitutional modifica reglas fundamentales del Blueprint o gobierno.

Ejemplos:

- Cambiar principios permanentes.
- Modificar autoridad.

- Cambiar reglas de aprobación.

Requerirá revisión reforzada y modificación explícita del documento gobernante.

21. Nivel de Impacto

Además de su categoría, una decisión podrá evaluarse como:

Low
Medium
High
Critical

Factores:

- Alcance.
 - Reversibilidad.
 - Seguridad.
 - Datos.
 - Operación.
 - Costo.
 - Dependencias.
 - Duración del impacto.
-

22. Decision Drivers

Una decisión arquitectónica relevante deberá considerar drivers reales.

Ejemplos:

- Business value.
- Maintainability.
- Security.
- Scalability.
- Reliability.
- Operability.
- Team capability.
- Cost.
- Simplicity.
- Evolution.

No deberán utilizarse todos indiscriminadamente.

23. Architecture Decision Flow

El flujo base será:

```
Problem identified
  ↓
Context understood
  ↓
Decision classification
  ↓
Alternatives
  ↓
Trade-offs
  ↓
Recommendation
  ↓
Review
  ↓
Decision
  ↓
ADR if required
  ↓
Blueprint / Standards update
  ↓
Implementation
```

24. Problem First

No deberá iniciarse una decisión con:

```
"We need technology X"
```

sin antes comprender el problema.

Formato correcto:

```
Problem
  ↓
Drivers
```

↓
Options
↓
Technology if needed

25. Decision Brief

Las decisiones de impacto Medium o superior podrán iniciar mediante un:

Decision Brief

que contenga:

- Problem.
- Context.
- Drivers.
- Constraints.
- Alternatives.
- Trade-offs.
- Recommendation.
- Open questions.

26. Decision Brief vs ADR

Un Decision Brief ayuda a tomar la decisión.

Un ADR conserva la decisión tomada.

Relación:

Decision Brief
Analysis

↓

Decision

↓

ADR
Historical record

Un Decision Brief no sustituye un ADR cuando éste sea requerido.

27. Architecture Review

Una Architecture Review evaluará una propuesta contra:

- Business context.
 - Architecture principles.
 - Blueprint.
 - Existing ADR.
 - Standards.
 - Quality attributes.
 - Risks.
 - Evolution.
-

28. Architecture Review Outcomes

Una revisión podrá resultar en:

Approved
Approved with Conditions
Changes Required
Deferred
Rejected

Estos resultados describen el review de una propuesta.

No sustituyen los estados documentales.

29. Approved

La propuesta puede continuar dentro de su alcance.

30. Approved with Conditions

La propuesta puede continuar si se cumplen condiciones explícitas.

Las condiciones deberán ser:

- Concretas.
 - Verificables.
 - Asignadas.
-

31. Changes Required

La propuesta necesita ajustes antes de aprobación.

32. Deferred

No existe suficiente información o no es el momento adecuado para decidir.

Deberá indicarse qué evento permitirá retomarla.

33. Rejected

La propuesta no deberá continuar.

Cuando el análisis tenga valor histórico podrá documentarse mediante ADR Rejected.

34. Cuándo se Requiere ADR

Se requerirá ADR cuando una decisión:

- Sea difícil de revertir.
- Afecte varias áreas.
- Establezca patrón importante.
- Seleccione una estrategia tecnológica significativa.
- Cambie una decisión previa.
- Genere consecuencias de largo plazo.
- Requiera preservar trade-offs.

35. Cuándo No se Requiere ADR

No deberá crearse ADR para:

- Decisiones triviales.
- Refactorizaciones locales.
- Aplicación directa de un Standard.
- Configuración rutinaria.
- Cambios reversibles de bajo impacto.

36. ADR Governance

Todo ADR deberá cumplir:

TPL-001 — ADR Template

La decisión no podrá marcarse como:

Accepted

sin aprobación humana correspondiente.

37. ADR Sequence

Los ADR se numerarán conforme sean necesarios.

No se reservarán ADR para decisiones hipotéticas.

38. Superseding Decisions

Cuando una decisión cambie sustancialmente:

Old ADR
Superseded

↓

New ADR
Accepted

No deberá reescribirse el pasado.

39. Architecture Baseline

Una Architecture Baseline representa el conjunto de decisiones aprobadas que definen un estado arquitectónico coherente.

Podrá incluir:

- Reference Architecture.
- ADR.
- Standards.
- Diagrams.
- Strategies.

La baseline deberá establecerse únicamente cuando exista suficiente conocimiento.

40. Baseline Change

Un cambio a una baseline deberá evaluar:

- Motivation.
- Impact.
- Compatibility.
- Migration.
- Risks.
- Affected documents.

No deberá modificarse implícitamente durante implementación.

41. Gates

El proyecto utilizará gates para proteger dependencias relevantes.

Un gate representa:

Una condición que debe satisfacerse antes de comenzar una actividad dependiente.

42. Tipos de Gate

Podrán existir:

Document Gate
Discovery Gate
Architecture Gate
Implementation Gate
Production Gate

43. Document Gate

Ejemplo:

RPC-003 Approved
↓
RPC-004 may start

44. Discovery Gate

Protege el inicio de una fase de descubrimiento.

Ejemplo futuro:

Business Discovery Guide Approved
↓
Business Discovery may start

45. Architecture Gate

Protege el diseño arquitectónico hasta contar con información suficiente.

46. Implementation Gate

Protege implementación contra decisiones prematuras.

47. Production Gate

Protegerá posteriormente el paso hacia producción.

Su definición detallada pertenecerá a documentos posteriores.

48. Gate Bypass

Un gate no deberá ignorarse informalmente.

Si existe una razón excepcional deberá:

1. Identificarse.
 2. Evaluarse riesgo.
 3. Definirse alcance.
 4. Autorizarse.
 5. Documentarse.
 6. Tener fecha o condición de cierre.
-

49. Trabajo Paralelo

El trabajo paralelo podrá permitirse cuando:

- No exista dependencia real.
 - La incertidumbre esté controlada.
 - Los outputs puedan evolucionar sin retrabajo significativo.
 - No se conviertan propuestas en decisiones prematuras.
-

50. Speculative Work

Podrá realizarse exploración especulativa cuando esté claramente identificada como:

Exploration

No deberá confundirse con:

Approved Design

51. Architecture Exception

Una Architecture Exception permite incumplir temporal o permanentemente una regla arquitectónica aprobada.

Deberá ser excepcional.

52. Excepciones No Permitidas Informalmente

No será válido:

“No cumplimos el Standard porque era más rápido.”

sin evaluación.

53. Contenido de una Excepción

Toda excepción relevante deberá incluir:

- Rule affected.
 - Reason.
 - Scope.
 - Risk.
 - Mitigation.
 - Owner.
 - Expiration or review condition.
 - Approval.
-

54. Temporary Exception

Una excepción temporal deberá tener:

- Fecha límite.

o:

- Condición de salida.

Ejemplo:

Valid until migration X is completed.

55. Permanent Exception

Una excepción permanente deberá justificarse cuidadosamente.

Si existen muchas excepciones permanentes a la misma regla, deberá revisarse la regla.

56. Exception Debt

Una excepción temporal pendiente constituye deuda arquitectónica controlada.

Deberá ser visible.

57. Architecture Debt

Architecture Debt representa una desviación conocida respecto del estado arquitectónico deseado.

Puede surgir por:

- Restricciones.
- Urgencia.
- Legacy.
- Decisiones temporales.
- Evolución incompleta.

58. Debt Is Not Failure

La existencia de deuda arquitectónica no implica automáticamente mala ingeniería.

La deuda no gestionada sí representa riesgo.

59. Architecture Debt Record

Una deuda significativa deberá identificar:

- Description.
- Reason.
- Impact.
- Owner.
- Priority.
- Exit strategy.

La forma específica podrá definirse mediante Guide, Checklist o herramienta posterior.

60. Debt Review

La deuda arquitectónica deberá revisarse periódicamente cuando exista suficiente volumen para justificarlo.

No deberá mantenerse invisiblemente.

61. Architecture Drift

Architecture Drift ocurre cuando la implementación se desvía de la arquitectura aprobada sin decisión explícita.

Ejemplos:

- Nuevas dependencias.
 - Patrones distintos.
 - Shared data no aprobado.
 - Integraciones improvisadas.
-

62. Drift Detection

Podrá detectarse mediante:

- Code Review.
- Architecture Review.
- Automated checks.
- Audits.
- Dependency analysis.

Los mecanismos concretos se definirán posteriormente.

63. Drift Response

Cuando se detecte drift deberá decidirse:

Implementation is wrong

o:

Architecture needs evolution

No deberá corregirse automáticamente uno u otro sin análisis.

64. Standard Governance

Los Standards:

STD

definirán reglas obligatorias.

Su creación deberá derivarse de:

- Arquitectura aprobada.
- Riesgos.
- Decisiones.
- Necesidad de consistencia.

65. Standard Approval

Todo Standard requerirá revisión y aprobación antes de convertirse en obligatorio.

66. Standard Exception

El incumplimiento deliberado de un Standard deberá seguir el proceso de excepción cuando el impacto lo amerite.

67. Guides Governance

Las Guides explicarán cómo aplicar principios o Standards.

No podrán crear silenciosamente obligaciones nuevas.

68. Checklist Governance

Las Checklists deberán verificar reglas existentes.

No deberán convertirse en autoridad primaria.

69. Prompt Governance

Los prompts deberán consumir reglas aprobadas.

Un PRM no podrá alterar una decisión arquitectónica.

70. Diagram Governance

Un diagrama deberá reflejar arquitectura existente.

No podrá introducir arquitectura únicamente mediante representación visual.

71. Change Proposal

Un cambio arquitectónico relevante deberá iniciar mediante una propuesta explícita.

La propuesta deberá explicar:

- Current state.
 - Problem.
 - Desired change.
 - Impact.
 - Risks.
 - Migration considerations.
-

72. Change Impact Analysis

Deberá evaluar, cuando corresponda:

- Business.
 - Domain.
 - Services.
 - APIs.
 - Data.
 - Security.
 - Operations.
 - Tests.
 - Documentation.
 - Cost.
 - Team.
-

73. Breaking Architectural Change

Un cambio será considerado breaking cuando invalide contratos o supuestos importantes.

Requerirá análisis adicional.

74. Incremental Change

Se favorecerán cambios incrementales cuando sea razonable.

Principio:

Evolve architecture deliberately rather than rewrite by default.

75. Architecture Review Cadence

No se establece inicialmente una reunión fija obligatoria.

Las Architecture Reviews deberán ocurrir cuando exista una decisión que lo justifique.

Esto evita ceremonias sin contenido.

76. Periodic Health Review

Cuando la plataforma alcance implementación significativa podrá establecerse una revisión periódica de salud arquitectónica.

Todavía no se define frecuencia.

77. Architecture Review Participants

Participarán únicamente los roles necesarios.

Podrán incluir:

- Chief Software Architect.
- Technical Lead.
- Relevant developers.
- Functional leadership.
- Specialists.

No deberá requerirse participación masiva por defecto.

78. Decision Owner

Toda decisión importante deberá tener un owner responsable de:

- Llevarla a análisis.
- Obtener información.
- Coordinar revisión.

- Asegurar documentación.

El owner no necesariamente es quien aprueba.

79. Decision Approver

El Approver será quien tenga autoridad para aceptar la decisión.

Para decisiones arquitectónicas de alto impacto será normalmente:

Chief Software Architect

con participación de otros responsables cuando el área lo exija.

80. Functional Approval

Cuando una decisión implique interpretación de negocio deberá existir validación funcional.

Arquitectura no deberá aprobar reglas funcionales por sí sola.

81. Security Approval

Decisiones críticas de seguridad podrán requerir revisión especializada.

La definición detallada será parte de Security Architecture.

82. Escalation

Una decisión deberá escalarse cuando:

- No exista consenso razonable.
 - Exista conflicto con Blueprint.
 - Tenga impacto superior al authority level del proponente.
 - Requiera excepción.
 - Existan riesgos no aceptados.
-

83. Conflict Resolution

El conflicto se resolverá mediante:

1. Clarificar el problema.
 2. Identificar drivers.
 3. Separar preferencias de restricciones.
 4. Comparar opciones.
 5. Revisar autoridad documental.
 6. Tomar decisión explícita.
 7. Registrar cuando corresponda.
-

84. Architecture Principles as Decision Tool

Los principios arquitectónicos futuros deberán utilizarse para decidir, no únicamente como declaraciones aspiracionales.

Cuando dos opciones sean viables, los principios ayudarán a evaluar cuál está más alineada.

85. Evidence

Las decisiones importantes deberán apoyarse en evidencia suficiente.

Podrá provenir de:

- Business Discovery.
 - Domain Discovery.
 - Documentation.
 - Proof of Concept.
 - Measurements.
 - External research.
 - Operational constraints.
-

86. Proof of Concept

Una PoC podrá utilizarse cuando exista incertidumbre técnica relevante.

Una PoC:

- Reduce incertidumbre.
 - No constituye automáticamente arquitectura final.
 - No deberá convertirse accidentalmente en producción.
-

87. Spike

Un Spike podrá utilizarse para investigación limitada.

Deberá tener:

- Question.
- Time/Scope boundary.
- Outcome.

No requiere necesariamente producción de código reusable.

88. Prototype

Un Prototype podrá explorar experiencia o diseño.

No deberá considerarse automáticamente solución productiva.

89. Research Result

Una investigación deberá distinguir:

Evidence

de:

Recommendation

y:

90. Decision Reopening

Una decisión Accepted podrá revisarse cuando:

- Cambien drivers.
 - Cambien restricciones.
 - Aparezca nueva evidencia.
 - La solución no cumpla objetivos.
 - Aparezca riesgo significativo.
-

91. Decision Stability

No deberá reabrirse una decisión únicamente por preferencia personal o tecnología nueva.

Deberá existir motivo material.

92. Architecture Consistency

Las decisiones deberán evaluarse también por su coherencia con el conjunto.

Una buena decisión aislada puede ser incorrecta si rompe el sistema global.

93. Local Optimization

Se evitarán decisiones que optimicen un componente a costa de:

- Acoplamiento global.
 - Operabilidad.
 - Seguridad.
 - Mantenibilidad.
-

94. Architecture Compliance

El cumplimiento deberá verificarse mediante mecanismos proporcionales.

Podrán incluir:

- Review.
 - Checklists.
 - Automated validation.
 - Tests.
 - Audits.
-

95. Compliance Is Not Conformance Theater

El objetivo no será demostrar que existe proceso.

El objetivo será demostrar que el sistema respeta decisiones importantes.

96. Architecture Audit

Una auditoría podrá evaluar:

- Drift.
 - Exceptions.
 - Debt.
 - ADR consistency.
 - Standards.
 - Dependency violations.
-

97. AI-Assisted Audit

Una IA podrá ayudar a detectar problemas.

Sus hallazgos requerirán interpretación humana antes de acciones significativas.

98. Governance Records

El gobierno deberá producir únicamente los registros necesarios.

Podrán incluir:

- ADR.
- Review findings.
- Exception records.
- Debt records.

No se crearán artefactos sin utilidad clara.

99. Meeting Notes

Una minuta no constituye decisión arquitectónica.

Cuando una reunión produzca una decisión, ésta deberá trasladarse al artefacto correspondiente.

100. Chat Decisions

Una decisión tomada en chat tampoco deberá permanecer únicamente allí.

Deberá documentarse según su importancia.

101. Pull Request Decisions

Una discusión de Pull Request podrá originar una decisión.

Si tiene alcance arquitectónico, deberá trasladarse fuera del PR hacia el Blueprint o ADR correspondiente.

102. Code as Evidence

El código puede evidenciar cómo está implementado el sistema.

No es automáticamente la fuente de la arquitectura deseada.

103. Emergency Governance

Ante una situación urgente podrá existir un proceso reducido.

Principios:

- Reducir proceso, no responsabilidad.
 - Registrar después lo necesario.
 - Evaluar deuda introducida.
 - Revisar la solución temporal.
-

104. Emergency Decision Flow

Conceptualmente:

```
Urgent risk
↓
Minimum safe decision
↓
Authorized execution
↓
Stabilize
↓
Post-review
↓
ADR / debt / correction if required
```

105. No Permanent Emergency

Una excepción urgente no deberá transformarse silenciosamente en arquitectura permanente.

106. Documentation Update

Una decisión aprobada deberá actualizar los documentos afectados.

Posibles artefactos:

- RPC.

- ADR.
 - STD.
 - GDE.
 - CHK.
 - PRM.
 - Diagrams.
-

107. Manifest Update

Cuando un nuevo documento gobernado sea creado o cambie de estado deberá actualizarse:

RPC-000 – Blueprint Manifest

108. Governance and Versioning

Los cambios deberán respetar el versionado establecido por META-000.

Un cambio arquitectónico sustancial podrá requerir nuevas versiones de documentos relacionados.

109. Governance and Deprecated Documents

Un documento Deprecated no deberá utilizarse como fuente vigente.

RPC-000 deberá señalar la fuente actual cuando exista.

110. Governance During Business Discovery

Business Discovery deberá operar sin introducir decisiones técnicas prematuras.

Architecture Governance deberá:

- Proteger alcance.
 - Facilitar resolución de dudas.
 - Registrar hallazgos relevantes.
 - Evitar traducir automáticamente capacidades a microservicios.
-

111. Governance During Domain Discovery

Durante Domain Discovery deberá protegerse:

- Ubiquitous Language.
- Evidencia.
- Hipótesis.
- Validación.

Candidatos a Bounded Context no deberán tratarse como definitivos sin revisión.

112. Governance During Architecture Design

Durante Architecture Design:

- Se analizarán alternativas.
 - Se aplicarán principios.
 - Se crearán ADR.
 - Se establecerán baselines.
 - Se identificarán Standards.
-

113. Governance During Implementation

Durante implementación:

- Architecture deberá consumirse, no reinventarse.
 - Las desviaciones deberán escalarse.
 - Los cambios significativos deberán volver al proceso de decisión.
-

114. Governance During Production

En fases futuras, incidentes y evidencia operativa podrán impulsar cambios arquitectónicos.

La producción será fuente de aprendizaje, no una excepción al gobierno.

115. Architecture Governance Anti-Patterns

El proyecto evitará:

Architecture Board for Everything

Enviar toda decisión menor a comité.

Architect as Bottleneck

Centralizar decisiones que podrían delegarse.

Architecture by Authority

Aprobar una opción únicamente porque la propone el rol de mayor jerarquía.

Architecture by Consensus

Esperar unanimidad absoluta para toda decisión.

Undocumented Exception

Desviarse de una regla sin registro.

ADR for Everything

Crear ADR para detalles triviales.

No ADR Ever

Mantener decisiones importantes sólo en conversaciones.

Process over Outcome

Optimizar cumplimiento ceremonial en vez de calidad.

Permanent Draft

Mantener decisiones relevantes indefinidamente sin resolver.

Technology First

Comenzar desde una herramienta y buscar posteriormente un problema.

Architecture Astronautics

Diseñar capacidades para escenarios hipotéticos extremos.

116. Decision Delegation Principle

La autoridad deberá situarse tan cerca del trabajo como sea razonablemente seguro.

Modelo:

Local
Developer

Tactical
Technical Lead

Architectural
Architecture Office

Strategic
Architecture + Relevant Leadership

Constitutional
Formal Blueprint Governance

Este modelo es orientativo y deberá considerar impacto real.

117. Governance Matrix

Decisión	Proponente típico	Review	Aprobación	ADR
Local	Developer	Peer / Lead	Developer/Lead	No
Tactical	Developer/Lead	Technical	Technical Lead	Opcional
Architectural	Lead/Architect	Architecture	Chief Software Architect	Frecuente
Strategic	Architect/ Leadership	Extended Architecture	Human Authority	Sí
Constitutional	Architecture Office	Formal Review	Project Governance	Sí / documento gobernante

118. Gate Matrix Inicial

Gate	Requisito
RPC-004	RPC-003 Approved
Business Discovery	RPC-004 Approved
Domain Discovery	Business Discovery suficiente + RPC-005 Approved
Architecture Design	Discovery suficiente + principios/gobierno aplicables
Implementation	Architecture baseline + Standards necesarios
Production	Production Readiness criteria

Los gates posteriores podrán refinarse conforme avance el Blueprint.

119. Estado Actual del Proyecto

Conforme al Manifest:

PROJECT_BOOTSTRAP
Frozen

META
Approved baseline

TPL
Initial set Approved

RPC-000
Approved

RPC-001
Approved

RPC-002
Approved

RPC-003
Draft

Por tanto:

Business Discovery
Not Started

Domain Discovery
Not Started

Architecture Design
Not Started

Implementation
Not Started

120. Criterios de Calidad

RPC-003 será adecuado si permite responder:

1. ¿Quién puede decidir qué?
2. ¿Qué decisiones requieren ADR?
3. ¿Cuándo se requiere revisión?
4. ¿Cómo se maneja una excepción?
5. ¿Cómo se cambia una decisión?
6. ¿Cómo se controla deuda?
7. ¿Cómo se detecta drift?
8. ¿Cómo funcionan los gates?
9. ¿Cuándo puede delegarse una decisión?

10. ¿Cómo evitamos que gobierno se convierta en burocracia?

121. Criterios de Aceptación

RPC-003 podrá pasar a Approved cuando:

- El modelo de autoridad sea aceptado.
 - La clasificación de decisiones sea validada.
 - El proceso de decisión sea aprobado.
 - Las reglas ADR sean aceptadas.
 - Los outcomes de Architecture Review sean validados.
 - El modelo de gates sea aprobado.
 - Las reglas de trabajo paralelo sean aceptadas.
 - El modelo de excepciones sea aprobado.
 - La gestión de Architecture Debt sea aceptada.
 - El manejo de Architecture Drift sea validado.
 - El modelo de delegación sea aprobado.
 - Los antipatronos sean aceptados.
 - No existan contradicciones con RPC-000, RPC-001 o RPC-002.
-

122. Restricciones de Fase

Mientras RPC-003 permanezca en Draft o In Review:

- No se iniciará RPC-004.
 - No se iniciará Business Discovery.
 - No se iniciará Domain Discovery.
 - No se definirán Bounded Contexts.
 - No se diseñarán microservicios.
 - No se definirá arquitectura técnica.
 - No se generará código productivo.
-

123. Próximo Gate

El gate actual será:

RPC-003
Architecture Governance

Draft

↓
Review
↓
Approved

Su aprobación habilitará:

RPC-004 — Business Discovery Guide

RPC-004 deberá aprobarse antes de iniciar formalmente Business Discovery.

124. Relación con otros documentos

Documento superior

- RPC-000 — Blueprint Manifest

Documentos gobernantes relacionados

- RPC-001 — AI Project Constitution
- RPC-002 — AI Collaboration Charter

Templates aplicables

- TPL-000 — Document Template Standard
- TPL-001 — ADR Template
- TPL-002 — Diagram Standards

Antecedentes fundacionales

- BOOT-003 — Architecture Vision
- BOOT-005 — Project Decisions
- BOOT-006 — Project Roadmap

Documento siguiente condicionado

- RPC-004 — Business Discovery Guide

Documentos futuros relacionados

- RPC-005 — Domain Modeling Guide
- RPC-006 — Architecture Principles
- RPC-007 — Reference Architecture

- ADR Series
 - STD Series
 - GDE Series
 - CHK Series
-

125. Principio Final

Architecture Governance seguirá:

Las decisiones deben ocurrir en el nivel más cercano posible al trabajo y en el nivel más alto necesario para proteger la arquitectura.

El buen gobierno no centraliza todas las decisiones.

Define:

- Límites.
- Autoridad.
- Escalamiento.
- Trazabilidad.

para que el equipo pueda avanzar con autonomía sin perder coherencia.

La arquitectura deberá ser gobernada lo suficiente para mantenerse saludable y lo suficientemente ligera para poder evolucionar.

126. Historial de Cambios

Versión	Fecha	Descripción
1.0	2026-07-24	Creación inicial de RPC-003 — Architecture Governance. Documento generado en estado Draft para revisión.

Estado

Draft — Pending Review

No se autoriza el inicio de **RPC-004 — Business Discovery Guide** ni de Business Discovery hasta completar la revisión y aprobación de RPC-003.